

OSSP-SKILL-WEEK8

SESSION-15

TASK1: Implement combined input and output redirection. The program should parse combined redirection, duplicate file descriptors, merge streams, verify ordering, handle edge cases, and test integrated outputs.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main(int argc, char *argv[])
{
    int input_fd;
    int output_fd;
    int saved_stdin;
    int saved_stdout;
    char buffer[200];
    ssize_t bytes;

    printf("Session 15 - Task 1: Combined Redirection\n");
    printf("-----\n");

    if (argc != 3)
    {
        printf("Usage: %s <input_file> <output_file>\n", argv[0]);
        return 1;
    }

    /* Open input file */
    input_fd = open(argv[1], O_RDONLY);

    if (input_fd == -1)
    {
        perror("Error opening input file");
        return 1;
    }

    /* Open output file */
    output_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_fd == -1)
```

```

if (output_fd == -1)
{
    perror("Error opening output file");
    close(input_fd);
    return 1;
}

/* Save original stdin and stdout */
saved_stdin = dup(STDIN_FILENO);
saved_stdout = dup(STDOUT_FILENO);

/* Redirect stdin */
if (dup2(input_fd, STDIN_FILENO) == -1)
{
    perror("dup2 stdin");
    return 1;
}

/* Redirect stdout */
if (dup2(output_fd, STDOUT_FILENO) == -1)
{
    perror("dup2 stdout");
    return 1;
}

close(input_fd);
close(output_fd);

printf("Combined redirection successful.\n");
printf("Input file: %s\n", argv[1]);
printf("Output file: %s\n", argv[2]);
printf("Data copied from input to output:\n");

while ((bytes = read(STDIN_FILENO,
                    buffer,
                    sizeof(buffer) - 1)) > 0)
{

```

```

while ((bytes = read(STDIN_FILENO,
                    buffer,
                    sizeof(buffer) - 1)) > 0)
{
    buffer[bytes] = '\0';
    printf("%s", buffer);
}

/* Restore stdin and stdout */
dup2(saved_stdin, STDIN_FILENO);
dup2(saved_stdout, STDOUT_FILENO);

close(saved_stdin);
close(saved_stdout);

printf("\nOriginal stdin and stdout restored successfully.\n");

return 0;

```

Output:

```

root@LAPTOP-1C0JKKG6:~# nano session15_task1.c
root@LAPTOP-1C0JKKG6:~# gcc session15_task1.c -o session15_task1
root@LAPTOP-1C0JKKG6:~# echo "Hello from the input file." > input15.txt
echo "This data is redirected." >> input15.txt
root@LAPTOP-1C0JKKG6:~# cat input15.txt
Hello from the input file.
This data is redirected.
root@LAPTOP-1C0JKKG6:~# ./session15_task1 input15.txt output15.txt
Session 15 - Task 1: Combined Redirection
-----

Original stdin and stdout restored successfully.
root@LAPTOP-1C0JKKG6:~# cat output15.txt
Combined redirection successful.
Input file: input15.txt
Output file: output15.txt
Data copied from input to output:
Hello from the input file.
This data is redirected.
root@LAPTOP-1C0JKKG6:~# |

```

Observation: I learned how to combine input and output redirection using file descriptors and `dup2()`. I understood how to preserve and restore the original standard streams while handling redirected data.

Task2: Implement a program to parse complex commands, combine pipes and redirections, resolve execution order, validate syntax, generate execution plans, and test different scenarios.

Source code:

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;
    int output_fd;

    printf("Session 15 - Task 2: Complex Command\n");
    printf("-----\n");
    printf("Command: cat input15.txt | grep data > result15.txt\n\n");

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* Create first process */
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        /* First command: cat input15.txt */

        close(pipefd[0]);

        /* Redirect stdout to pipe */
        dup2(pipefd[1], STDOUT_FILENO);
```

```
dup2(pipefd[1], STDOUT_FILENO);

close(pipefd[1]);

execlp("cat", "cat", "input15.txt", NULL);

perror("cat execution failed");
exit(1);
}

/* Create second process */
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    /* Second command: grep data */

    close(pipefd[1]);

    /* Redirect stdin from pipe */
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    /* Open output file */
    output_fd = open("result15.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd == -1)
    {
        perror("output file");
        exit(1);
    }
}
```

```

    close(pipefd[0]);

    /* Open output file */
    output_fd = open("result15.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    if (output_fd == -1)
    {
        perror("output file");
        exit(1);
    }

    /* Redirect stdout to result file */
    dup2(output_fd, STDOUT_FILENO);

    close(output_fd);

    execlp("grep", "grep", "data", NULL);

    perror("grep execution failed");
    exit(1);
}

/* Parent closes pipe descriptors */
close(pipefd[0]);
close(pipefd[1]);

/* Wait for both processes */
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Complex command executed successfully.\n");
printf("Result stored in result15.txt\n");

return 0;
}

```

Output:

```
root@LAPTOP-1COJKKG6:~# nano session15_task2.c
root@LAPTOP-1COJKKG6:~# gcc session15_task2.c -o session15_task2
root@LAPTOP-1COJKKG6:~# cat input15.txt
Hello from the input file.
This data is redirected.
root@LAPTOP-1COJKKG6:~# ./session15_task2
Session 15 - Task 2: Complex Command
-----
Command: cat input15.txt | grep data > result15.txt

Complex command executed successfully.
Result stored in result15.txt
root@LAPTOP-1COJKKG6:~# cat result15.txt
This data is redirected.
root@LAPTOP-1COJKKG6:~# |
```

Observation: I learned how to combine pipes with redirection to execute complex commands. I understood how processes communicate through pipes, how output can be redirected to a file, and how process execution order is maintained using `waitpid()`.

Session-16

Task1: Implement background job handling in a shell. The program should detect background jobs, launch non-blocking processes, return the prompt immediately, store job information, monitor execution, and test background tasks.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Session 16 - Task 1: Background Jobs\n");
    printf("-----\n");
    printf("Use: sleep 5 &\n");
    printf("Use: exit\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }

        int length = strlen(command);

        /* Check for background operator & */
        if (length > 0 && command[length - 1] == '&')
        {
            command[length - 1] = '\0';

            /* Remove trailing spaces */
            while (length > 1 &&
```

```

        while (length > 1 &&
               command[length - 2] == ' ')
        {
            command[length - 2] = '\0';
            length--;
        }

        pid = fork();

        if (pid < 0)
        {
            perror("fork");
        }
        else if (pid == 0)
        {
            execlp(command, command, NULL);

            perror("Command execution failed");
            exit(1);
        }
        else
        {
            printf("[Background job started] PID: %d\n", pid);
        }
    }
    else
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork");
        }
        else if (pid == 0)
        {
            execlp(command, command, NULL);

            perror("Command execution failed");
            exit(1);
        }
        else

```

```

        {
            waitpid(pid, NULL, 0);
            printf("Foreground job completed.\n");
        }
    }

    return 0;
}

```

Output:

Session 16 - Task 1: Background Jobs

Use: sleep 5 &
Use: exit

```
myshell> sleep 5 &  
[Background job started] PID: 1111  
myshell> sleep 2 &  
[Background job started] PID: 1114  
myshell> exit  
Shell exiting...  
root@LAPTOP-1COJKKG6:~# |
```

Observation: I learned how background jobs work in a shell and how `fork()` can be used to launch a non-blocking process. I understood how the shell can return the prompt immediately while the background process continues executing.

Task2: Implement a job table in a shell. The program should create a job table, store process groups, track job states, update status, remove completed jobs, and validate job records.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Running");

    printf("[%d] %d Running %s\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           jobs[job_count].command);
}
```

```
    job_count++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result > 0)
        {
            strcpy(jobs[i].state, "Completed");
        }
    }
}

void show_jobs()
{
    update_jobs();

    printf("\nJob Table\n");
    printf("-----\n");

    if (job_count == 0)
    {
        printf("No jobs available.\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d %-10s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
```

```

        jobs[i].state,
        jobs[i].command);
    }
}

void remove_completed_jobs()
{
    for (int i = 0; i < job_count;)
    {
        if (strcmp(jobs[i].state, "Completed") == 0)
        {
            for (int j = i; j < job_count - 1; j++)
            {
                jobs[j] = jobs[j + 1];
            }

            job_count--;
        }
        else
        {
            i++;
        }
    }
}

int main()
{
    char command[100];

    printf("Session 16 - Task 2: Job Table\n");
    printf("-----\n");
    printf("Use: sleep 5 &\n");
    printf("Use: jobs\n");
    printf("Use: clean\n");
    printf("Use: exit\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

```

```
if (fgets(command, sizeof(command), stdin) == NULL)
    break;

command[strcspn(command, "\n")] = '\0';

if (strcmp(command, "exit") == 0)
{
    printf("Shell exiting...\n");
    break;
}

else if (strcmp(command, "jobs") == 0)
{
    show_jobs();
}

else if (strcmp(command, "clean") == 0)
{
    update_jobs();
    remove_completed_jobs();

    printf("Completed jobs removed.\n");
}

else
{
    int length = strlen(command);

    if (length > 0 &&
        command[length - 1] == '&')
    {
        command[length - 1] = '\0';

        while (length > 1 &&
            command[length - 2] == ' ')
        {
            command[length - 2] = '\0';
            length--;
        }

        pid_t pid = fork();
```

```

        if (pid < 0)
        {
            perror("fork");
        }
        else if (pid == 0)
        {
            execlp(command, command, NULL);

            perror("Command failed");
            exit(1);
        }
        else
        {
            add_job(pid, command);
        }
    }
    else
    {
        printf("Use '&' for a background job.\n");
    }
}

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session16_task2.c
root@LAPTOP-1COJKKG6:~# gcc session16_task2.c -o session16_task2
root@LAPTOP-1COJKKG6:~# ./session16_task2
Session 16 - Task 2: Job Table
-----
Use: sleep 5 &
Use: jobs
Use: clean
Use: exit

myshell> sleep 5 &
[1] 1173 Running sleep 5
myshell> Command failed: No such file or directory
jobs

Job Table
-----
[1] PID=1173 Completed sleep 5
myshell> |

```

Observation: I learned how to create and maintain a job table for background processes. I understood how to track process IDs and job states, update completed jobs using `waitpid()` with `WNOHANG`, and remove completed jobs from the table

